

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**Báo cáo môn học:
Các vấn đề hiện đại Công nghệ thông tin**

**SoccerRAG: Multimodal Soccer Information
Retrieval via Natural Queries**

Giảng viên: TS. Nguyễn Đức Anh

Mã lớp môn học: INT3507 7

Nhóm: 4

Thành viên:

Nguyễn Hữu Đồng – 21020760

Lê Minh Châu – 21020286

Nguyễn Cao Đức – 21020762

Đặng Tiến Dũng – 21020753

Hà Nội – 2024

Phân công công việc

STT	Thành viên	MSV	Phân công công việc	Đóng góp
1	Nguyễn Hữu Đồng (Nhóm trưởng)	21020760	Đọc các bài được tìm để chọn chủ đề. Tìm hiểu phần đánh giá trong bài nghiên cứu, tìm hiểu code và chạy code phần database. Làm slide, viết báo cáo các phần tương ứng. Tìm hiểu các phần trong mục phụ lục	25%
2	Lê Minh Châu	21020286	Tìm các bài nghiên cứu để chọn chủ đề. Tìm hiểu phần giới thiệu và thảo luận trong bài nghiên cứu. Tìm hiểu code và chạy code phần extractor. Làm slide, viết báo cáo các phần tương ứng. Thực nghiệm các phần trong phụ lục	25%
3	Đặng Tiến Dũng	21020753	Đọc các bài được tìm để chọn chủ đề. Tìm hiểu background and related work và phần kết luận trong bài nghiên cứu. Tìm hiểu code và chạy code phần validator. Làm slide, viết báo cáo các phần tương ứng. Thực nghiệm các phần trong phụ lục	25%
4	Nguyễn Cao Đức	21020762	Đọc các bài được tìm để chọn chủ đề. Tìm hiểu phần kiến trúc chung trong bài nghiên cứu. Tìm hiểu và chạy code phần SQL Agent. Làm slide, viết báo cáo các phần tương ứng. Viết báo cáo các phần trong phụ lục.	25%

Mục lục

Phân công công việc	ii
Mục lục	iii
Danh mục hình vẽ	iv
Danh mục bảng biểu	v
1. Giới thiệu	1
1.1. Bối cảnh	1
1.2. Khó khăn	1
1.3. Đóng góp	2
2. Background và Related work	3
2.1. Thuật ngữ cơ bản về bóng đá	3
2.2. Dữ liệu bóng đá đa phương tiện	3
2.3. Các mô hình ngôn ngữ lớn LLMs	3
2.4. Retrieval Augmented Generation	4
2.5. LangChain và ChainLit	5
2.6. Các nghiên cứu liên quan	5
3. Kiến trúc đề xuất	6
3.1. Database	7
3.2. Feature Extractor	8
3.3. Feature Validator	9
3.4. SQL Agent	10
4. Đánh giá	12
4.1. Đánh giá bộ Extractor-Validator	13
4.2. Độ phức tạp câu truy vấn	13
4.3. Ablation Study	14
4.4. Thời gian thực thi	15
5. Thảo luận	16
5.1. Tầm quan trọng của xử lý Input	16
5.2. Truy xuất thống kê	16
5.3. Ảnh hưởng của LLM	17
5.4. Dự định phát triển	17
6. Kết luận	19
7. Phụ lục	20
7.1. Câu 1: Kết quả mô hình chạy đối với câu hỏi tiếng Việt.	20
7.2. Câu 2: Kết quả mô hình đối với các câu hỏi nhiều đối tượng	24
7.3. Câu 3: Sử dụng RAG trong phần SQL Agent	25
Cam kết không sử dụng công cụ AI trong quá trình viết báo cáo	27
Tài liệu tham khảo	28

Danh mục hình vẽ

Hình 2.1: Kiến trúc RAG	4
Hình 3.1: Kiến trúc tác giả đề xuất	6
Hình 3.2: Schema cho cơ sở dữ liệu SQLite	7
Hình 3.3: Module Feature Extractor	8
Hình 3.4: Ví dụ xử lý Feature Extractor	8
Hình 3.5: Module Feature Validator	9
Hình 3.6: Ví dụ xử lý của module Feature Validator	10
Hình 3.7: Module SQL agent	11
Hình 4.1: Tổng thời gian trả lời câu hỏi 1 đến 10 với GPT-3.5-Turbo và GPT 4.0-Turbo.	15

Danh mục bảng biểu

Bảng 4.1: Kết quả đánh giá chuỗi extractor-validator	13
Bảng 4.2: Kết quả đánh giá độ phức tạp query	14
Bảng 4.3: Kết quả Ablation study	14

Chương 1.

Giới thiệu

1.1. Bối cảnh

Trong bối cảnh hiện nay, dữ liệu thể thao trực tuyến ngày càng đa dạng và phát triển mạnh mẽ. Các nội dung thể thao trực tuyến bao gồm các bản phát sóng trận đấu, bình luận trận đấu, các sự kiện diễn ra như bàn thắng, thẻ đỏ,..., và các thông tin liên quan. Lượng thông tin khổng lồ đó đã kéo theo nhu cầu phát triển hệ thống có thể hiểu và xử lý câu truy vấn bằng ngôn ngữ tự nhiên. Nhờ hệ thống truy vấn trực quan, người dùng có thể đơn giản hóa các thao tác khi đặt câu hỏi cũng như được tiếp cận nhanh chóng với nguồn dữ liệu thể thao phong phú.

1.2. Khó khăn

Tuy nhiên, việc sử dụng nguồn dữ liệu truy vấn được trích xuất từ các broadcast bóng đá có thể dẫn đến một số những khó khăn [1] có thể kể đến như:

1. Câu truy vấn từ người dùng có chứa lỗi:

Một trong những yếu tố ảnh hưởng đến chất lượng của hệ thống truy vấn là việc các câu hỏi từ người dùng bị lỗi, ví dụ như lỗi chính tả, lỗi cấu trúc ngữ pháp, lỗi viết tắt,... . Các hệ thống truy vấn được phát triển trước đây thường tập trung vào các câu hỏi từ người dùng sử dụng từ ngữ được chọn lọc, viết rõ ràng thông tin các đối tượng.

Tuy nhiên, trên thực tế, người dùng bình thường sẽ có thói quen viết tắt hoặc viết sai lỗi chính tả, ngữ pháp. Một hệ thống hiệu quả cần phải xác định được lỗi từ câu truy vấn và sửa lỗi để đảm bảo rằng ý tứ của người dùng không bị ảnh hưởng.

2. Khả năng đáp ứng của hệ thống khi database thay đổi:

Dữ liệu bóng đá có tính chất thay đổi thêm mới thường xuyên. Đối với dữ liệu được cập nhật liên tục, việc database thay đổi có thể dẫn đến việc phải thay đổi cả mã nguồn hoặc kiến trúc hệ thống. Một hệ thống truy vấn hiệu quả yêu cầu có khả năng đáp ứng được các thay đổi từ dữ liệu mà lại giảm thiểu được việc bảo trì mã nguồn.

3. Khả năng truy vấn thông tin từ cơ sở dữ liệu từ ngôn ngữ tự nhiên:

Đối với lượng dữ liệu khổng lồ, hệ thống cần tổ chức lưu trữ trong các structured database. Để truy vấn thông tin từ database cần sử dụng các câu lệnh phù hợp và đúng quy tắc. Thách thức đặt ra là khả năng hệ thống hiểu và chuyển đổi các câu hỏi bằng ngôn ngữ tự nhiên từ người dùng, thành các câu lệnh truy vấn cơ sở dữ liệu chính xác. Điều này yêu cầu hệ thống cần có thể phân tích ngữ nghĩa, xử lý ngôn ngữ tự nhiên và chuyển thành câu lệnh truy vấn theo đúng mục đích của người dùng

1.3. Đóng góp

Bài báo đề xuất framework SoccerRAG, hệ thống truy xuất thông tin, tiên phong trong việc ứng dụng sức mạnh của RAG [2], SQL Agent [3] và LLM [4] để có thể truy vấn các thông tin liên quan đến trận đấu bóng đá như “Cầu thủ B có nhận được thẻ vàng không?” bằng ngôn ngữ tự nhiên.

Cụ thể, đóng góp chính của bài báo bao gồm:

1. Đề xuất và thiết kế framework SoccerRAG [5].

Đây là đề xuất mới, có khả năng truy xuất thông tin đa phương tiện các nội dung về trận đấu bóng đá. Framework này bao gồm các thành phần như biểu diễn dữ liệu, trích xuất và xác thực các đặc trưng và truy vấn trên cơ sở dữ liệu.

2. Cung cấp mã nguồn mở của SoccerRAG

Mã nguồn mở của RAG được cung cấp bởi nhà với hướng dẫn rõ ràng, chi tiết. Việc này đã hỗ trợ nhóm rất nhiều trong việc chạy thực nghiệm với hệ thống trong thời gian tìm hiểu và thử nghiệm.

3. Thực hiện đánh giá toàn diện

Hệ thống được thực hiện đánh giá trên nhiều phương diện như đánh giá thành phần trích xuất và xác thực đặc trưng, đánh giá độ phức tạp câu truy vấn, đánh giá đóng góp của các thành phần trong hệ thống riêng lẻ, đánh giá thời gian thực thi khi sử dụng mô hình LLM khác nhau.

Chương 2.

Background và Related work

2.1. Thuật ngữ cơ bản về bóng đá

Bóng đá là môn thể thao vua trên nhiều quốc gia trên thế giới. Đây là môn thể thao khá đơn giản, bao gồm 2 đội và đội nào đưa được bóng vào lưới đối phương nhiều hơn sẽ là đội chiến thắng trận đấu đó. Các trận đấu bóng đá cần tuân theo một bộ luật được thống nhất từ trước, nhằm đảm bảo công bằng cho các cầu thủ tham gia thi đấu.

Người điều hành trận đấu được gọi là trọng tài. Hiện nay trọng tài cũng có thể sử dụng những công nghệ hiện đại như VAR để có thể hỗ trợ đưa ra quyết định hiệu quả hơn và công bằng hơn. Trong một trận đấu có thể nhiều sự kiện xảy ra như những cú dứt điểm, thẻ phạt, sự thay người,...

2.2. Dữ liệu bóng đá đa phương tiện

Một trận đấu bóng đá phục vụ khán giả cả về phần nhìn và phần nghe. Ví dụ như bình luận của bình luận viên ở dưới dạng audio, trận bóng dưới dạng video, thông tin về số bàn thắng, số thẻ phạt thì ở dưới dạng text. Việc chỉ dùng văn bản để mô tả có thể khiến mô hình cứng nhắc và không linh hoạt. Để cải thiện tính linh hoạt, mô hình có thể sử dụng nhiều dữ liệu đa phương tiện khác như hình ảnh, âm thanh. Sự kết hợp này mang lại những hiểu biết vượt trội hơn, giúp công cụ mạnh mẽ hơn, toàn diện hơn, thông minh hơn so với các phương thức khác chỉ sử dụng văn bản riêng lẻ.

SoccerNet [6] là một tập dữ liệu thể thao đa phương tiện quy mô lớn chứa các video bóng đá, bao gồm 550 trận đấu được phát sóng trên các giải đấu hàng đầu châu Âu như Premier League, La Liga và Bundesliga. Bộ dữ liệu SoccerNet đặt ra rất nhiều thách thức liên quan, chính vì thế đã thu hút đông đảo cộng đồng các nhà nghiên cứu từ nhiều lĩnh vực khác nhau, bao gồm thị giác máy tính, học máy và phân tích thể thao. Các cuộc thi được tổ chức hàng năm đã cung cấp các phương pháp tiếp cận sáng tạo, nhằm phân tích bóng đá dựa trên tập dữ liệu khác nhau từ bộ dữ liệu. [7], [8].

2.3. Các mô hình ngôn ngữ lớn LLMs

Large Language Models (LLMs) là những mô hình được huấn luyện trên một tập dữ liệu văn bản khổng lồ [4]. Điều này giúp chúng có thể hiểu ngôn ngữ tự nhiên thật kỹ, từ đó phân tích và sinh ra văn bản một cách tự nhiên như con người. Sự phát triển gần đây của các mô hình LLMs

đã mang lại sự phát triển mạnh mẽ trong lĩnh vực xử lý ngôn ngữ tự nhiên [9]. Các mô hình này còn có thể lập luận, đưa ra quyết định và kết hợp cả với các dữ liệu đa phương tiện khác.

Mỗi mô hình ngôn ngữ lớn được phát triển dựa trên nhiều thành phần như token, chain hay vector database. Ngoài ra còn có các ứng dụng LLMs như các agents.

Token là các đơn vị ngôn ngữ cơ bản mà mô hình sử dụng cho quá trình tạo và dự đoán văn bản tiếp theo. Các token được phân tích ra từ các đoạn văn hay các câu thành phần nhỏ hơn như một câu, từ hay kí tự. Chúng đóng vai trò quan trọng trong việc tính toán chi phí API [10].

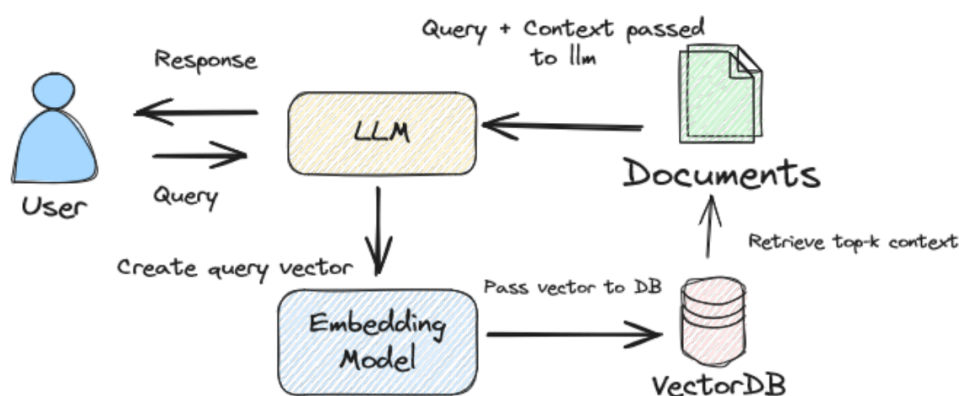
Chain là một chuỗi nhiệm vụ thực hiện theo thứ tự đã được định sẵn trước đó nhằm đảm bảo rằng các bước được tiến hành tuần tự và chính xác để có thể hoàn thành một tác vụ nhất định [11].

Agent [12] là tác nhân sử dụng LLM để suy luận, xác định những hành động

Vector Database là một hệ thống lưu trữ dữ liệu được tối ưu hóa để lưu trữ và truy vấn các vector (các tập hợp các giá trị số, thường là các số thực, được sắp xếp theo một thứ tự cố định). Chúng cho phép xử lý dữ liệu nhiều chiều, chẳng hạn như các vector từ văn bản và hình ảnh. Cơ sở dữ liệu vector thực hiện tìm kiếm tương tự nhanh chóng nhờ thuật toán ANN (các thuật toán dùng để tìm kiếm các điểm dữ liệu gần nhất trong không gian có kích thước lớn một cách hiệu quả) [13].

2.4. Retrieval Augmented Generation

Để truy vấn thông tin ngoài thì bài báo lựa chọn **Retrieval Augmented Generation (RAG)**. Phương pháp này là sự kết hợp giữa truy xuất thông tin văn bản và mô hình ngôn ngữ sinh văn bản để tạo ra câu trả lời phù hợp với ngữ cảnh [2]. RAG được ứng dụng trong nhiều lĩnh vực khác nhau để làm chatbot trả lời các câu hỏi dựa trên tài liệu nội bộ và cơ sở tri thức. Kiến trúc RAG cơ bản có 2 module chính, IR và Generation.



Hình 2.1: Kiến trúc RAG

Information Retrieval (IR) được gọi là bộ truy xuất thông tin, tập trung vào việc tìm kiếm dữ liệu có liên quan từ các tập dữ liệu khổng lồ dựa trên các keyword từ câu hỏi của người dùng. IR bao gồm nhiều kỹ thuật khác nhau như lập chỉ mục, thuật toán tìm kiếm, và xử lý ngôn ngữ tự nhiên, nhằm tổ chức và truy xuất dữ liệu không có cấu trúc một cách hiệu quả [14].

Generation [15] dựa trên những thông tin đã tìm kiếm được thông qua phân trích xuất dữ liệu ở trên để đưa vào mô hình ngôn ngữ lớn. Các mô hình sử dụng những dữ liệu này để có thể tạo ra câu trả lời tự nhiên, cụ thể và phù hợp với câu hỏi của người dùng.

2.5. LangChain và ChainLit

LangChain [16] là một framework mã nguồn mở, sử dụng các mô hình ngôn ngữ lớn để thực hiện các tác vụ như tạo ra câu trả lời, lập luận và đưa ra quyết định dựa trên một quy trình nhất định. Điểm mạnh của LangChain nằm ở khả năng tích hợp dữ liệu từ nhiều nguồn bên ngoài như cơ sở dữ liệu, API hoặc các nguồn thông tin khác, từ đó mở rộng khả năng của mô hình ngôn ngữ lớn. Framework này không chỉ giới hạn ở việc xử lý văn bản mà còn hướng tới xây dựng các ứng dụng thông minh có thể xử lý ngữ cảnh phức tạp, linh hoạt phản hồi và hỗ trợ giải quyết các vấn đề trong nhiều lĩnh vực khác nhau.

ChainLit [17] là một framework được sử dụng với mục đích xây dựng giao diện người dùng dễ dàng và nhanh chóng hơn dành cho các ứng dụng mô hình ngôn ngữ lớn dùng để trả lời câu hỏi. ChainLit phù hợp cho các trường hợp sử dụng như trợ lý ảo, chatbot, hệ thống hỏi đáp và các ứng dụng khác cần sự tương tác với người dùng.

2.6. Các nghiên cứu liên quan

Bài báo “**SoccerNet: A Scalable Dataset for Action Spotting in Soccer Videos**”

Bài báo của nhóm tác giả S. Giancola và cộng sự trình bày trong hội nghị IEEE Conference on Computer Vision and Pattern Recognition Workshop (CVPR) vào năm 2018 giới thiệu về bộ dữ liệu SoccerNet. Đây là một bộ dữ liệu quy mô lớn dành cho các nghiên cứu về lĩnh vực bóng đá đặc biệt là những hoạt động, sự kiện trong trận đấu. Mục tiêu của bài báo là cung cấp nguồn dữ liệu phong phú và đa dạng cho các nghiên cứu liên quan đến bóng đá.

Bài báo “**Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**”

Bài báo của nhóm tác giả P. Lewis và cộng sự trình bày trong hội nghị Advances in Neural Information Processing Systems (NeurIPS) năm 2021 nói về một phương pháp mới là Retrieval-Augmented Generation (RAG) để giải quyết các bài toán xử lý ngôn ngữ tự nhiên đòi hỏi những kiến thức chuyên sâu hoặc tài liệu nội bộ. Phương pháp này nhằm cải thiện khả năng trả lời các truy vấn phức tạp mà các mô hình ngôn ngữ lớn truyền thống gặp do hạn chế bộ nhớ và ngữ cảnh.

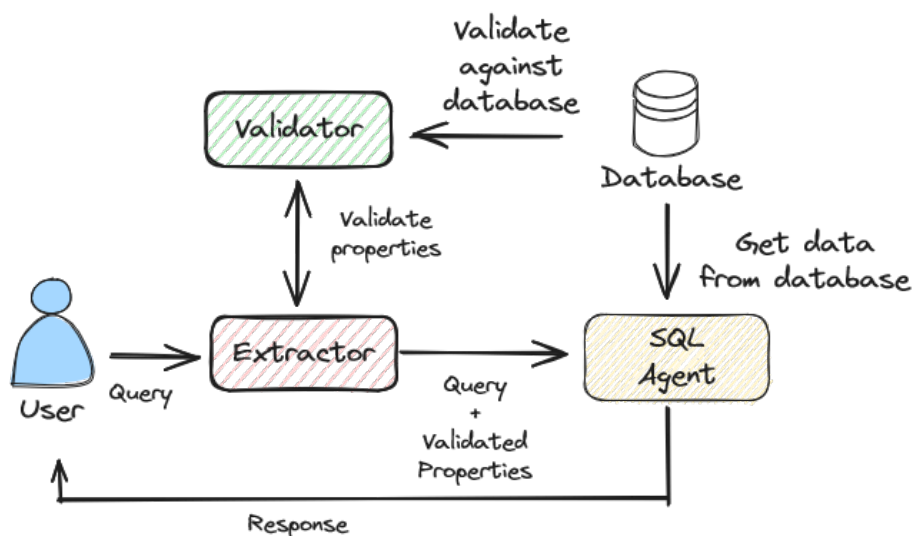
Bài báo “**AgentBench: Evaluating LLMs as Agents**”

Bài báo của nhóm tác giả X. Liu và cộng sự giới thiệu về AgentBench. Đây là một phương pháp đánh giá các agents của những mô hình ngôn ngữ lớn qua các vai trò như hiểu, lên kế hoạch trong các môi trường phức tạp và có tính tương tác.

Chương 3.

Kiến trúc đề xuất

Dưới đây là tổng quan kiến trúc của SoccerRAG, bao gồm 4 thành phần chính: database, Feature Extractor [18], Feature Validator, và SQL agent.



Hình 3.1: Kiến trúc tác giả đề xuất

Luồng hoạt động cơ bản của hệ thống SoccerRAG:

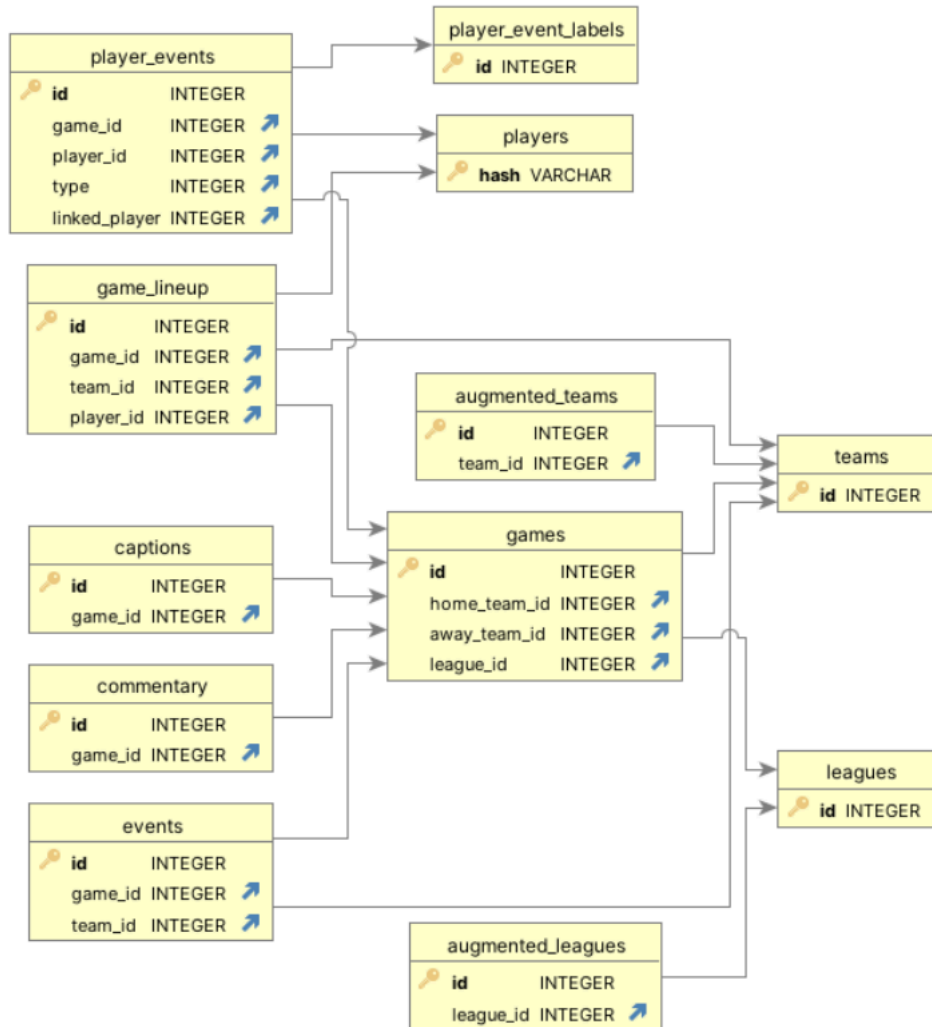
- Người dùng nhập câu hỏi bằng ngôn ngữ tự nhiên để truy vấn thông tin liên quan đến nội dung có trong database.
- Câu hỏi của người dùng sẽ được gửi đến LLM cùng với một schema được định nghĩa sẵn để hướng dẫn LLM trích xuất thông tin theo các mục có trong schema. Sau đó, LLM sẽ trả về danh sách các thuộc tính được trích xuất.
- Mỗi thuộc tính được trích xuất ở bước trên sẽ được kiểm tra lại một lần nữa nhằm sửa lỗi chính tả hoặc xử lý viết tắt.
- Prompt sau khi được xử lý sẽ được sử dụng để hướng dẫn LLM tạo truy vấn SQL, từ đó kết hợp trả lời câu hỏi của người dùng.

Qua các bước trên, hệ thống SoccerRAG đảm bảo quy trình xử lý một cách chính xác từ truy vấn ngôn ngữ tự nhiên câu hỏi của người dùng đến việc tạo truy vấn SQL và sau cùng là đưa ra được câu trả lời hoàn chỉnh.

3.1. Database

Hệ thống SoccerRAG sử dụng tập dữ liệu SoccerNet, đây là tập dữ liệu bao gồm các video phát sóng và âm thanh bình luận của 550 trận đấu bóng đá từ top các giải vô địch hàng đầu châu Âu [6]. Nhóm tác giả sử dụng Whisper của OpenAI [19] để xử lý dữ liệu âm thanh, kết quả thu được là các bình luận dạng văn bản.

Tuy nhiên, dữ liệu không có cấu trúc thường gây khó khăn trong việc truy xuất thông tin sử dụng RAG, thêm vào đó để chuyển dữ liệu chưa được xử lý sang dạng SQL cũng khá khó khăn. Vì thế, tác giả sử dụng thư viện SQLAlchemy [20] để chuyển đổi dữ liệu sang cơ sở dữ liệu SQLite với cấu trúc theo các bảng được định nghĩa sẵn.

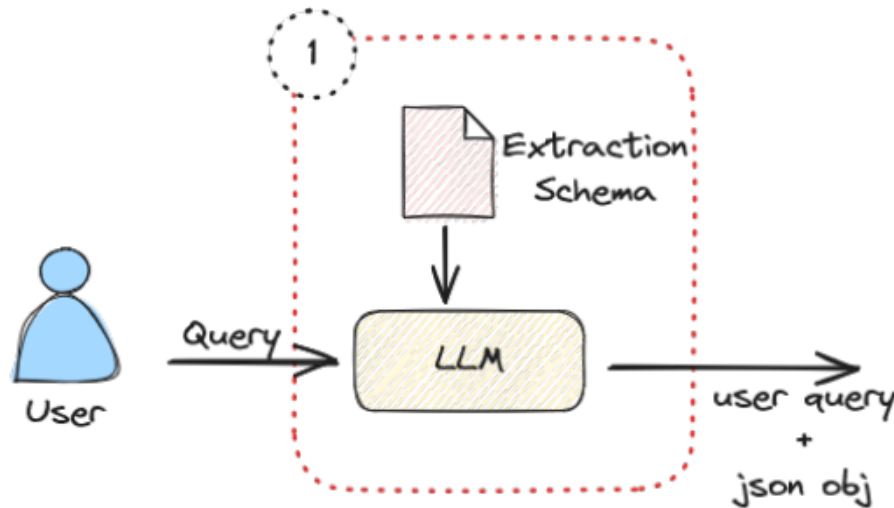


Hình 3.2: Schema cho cơ sở dữ liệu SQLite

Quy trình này đảm bảo rằng tất cả thông tin có liên quan đều được lưu trữ và dễ dàng truy cập đối với SQL Agent. Bằng cách cấu trúc dữ liệu theo cách này, việc truy xuất và tạo thông tin từ dữ liệu trở nên hiệu quả và chính xác hơn. Hình 3.2 minh họa biểu đồ quan hệ thực thể của cơ sở dữ liệu. Cấu trúc cơ bản của cơ sở dữ liệu gồm có các bảng sau: *teams*, *players*, *leagues*, *games*, *game_lineup*, *events*, *player_events*, *captions*, *commentary*, *player_events*, *player_events_labels*, *agumented_teams*, *agumented_leagues*.

3.2. Feature Extractor

Module Feature Extraction được sử dụng để trích xuất các đặc trưng trong câu hỏi của người dùng. Đây là một bước quan trọng để cải thiện chất lượng tổng thể của truy vấn SQL. Trên thực tế, mô hình chỉ cần một số đặc trưng nhất định từ câu hỏi từ người dùng để có thể trả lời được.



Hình 3.3: Module Feature Extractor

Hiểu được vấn đề đó, tác giả quyết định sử dụng module Feature Extractor để trích xuất những đặc trưng cần thiết phục vụ cho các bước tiếp theo. Đầu tiên, truy vấn của người dùng sẽ được đưa qua một LLM, kết hợp với một schema được định nghĩa trước, để chỉ trích xuất thông tin theo khung hướng dẫn. Đầu ra của module là các đặc trưng được liệt kê dưới dạng json và câu hỏi từ người dùng.

Input

How many goals did Messi score for Barca in the 15-16 season?

Schema

```
schema = {
  "properties": {
    "person_name": {"type": "string"},
    "team_name": {"type": "string"},
    "in_game_event": {"type": "string"},
    "season": {"type": "string"},
  },
  "required": (),
}
```

→

```
{ "person_name": "Messi", "team_name": "Barca",
  "in_game_event": "goal", "season": "15-16" }
```

Function

```
"name": "information_extraction",
"description": "Extracts information from the passage.",
"parameters": {"properties"}
```

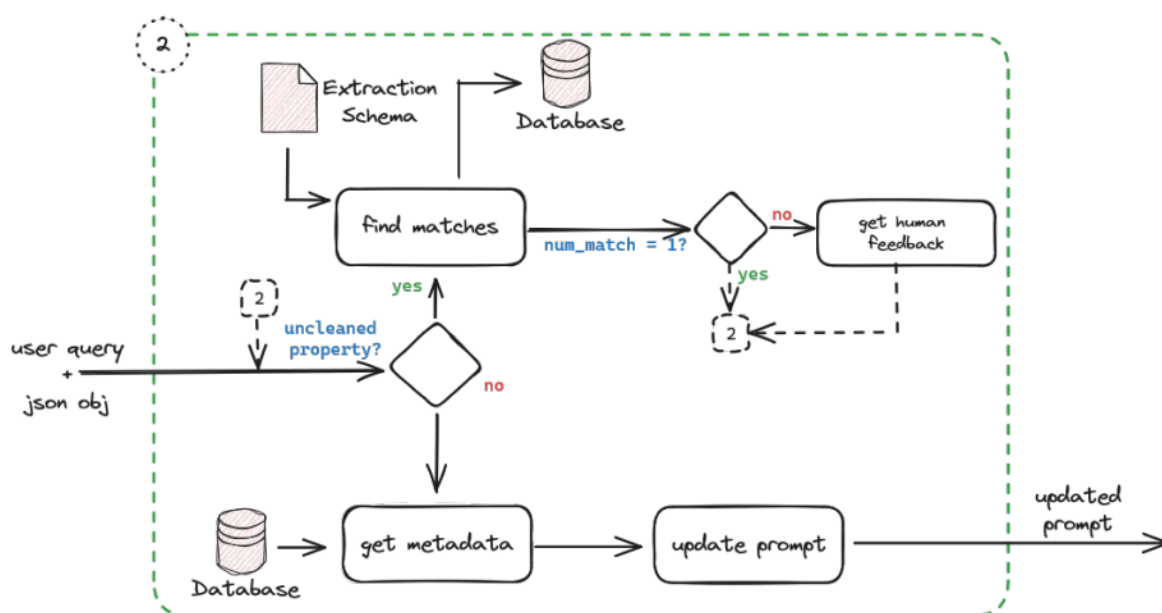
Hình 3.4: Ví dụ xử lý Feature Extractor

Trong ví dụ ở trên, từ câu truy vấn “How many goals did Messi score for Barca in the 15-16 season?”, LLM kết hợp với schema được định nghĩa gồm các thuộc tính là *person_name*, *team_name*, *in_game_event* và *season*, hệ thống chỉ trích xuất những đặc trưng ứng với các thuộc

tính đó. Các đặc trưng đã được trích xuất sẽ được sử dụng để làm đầu vào cho module tiếp theo là Feature Validator.

3.3. Feature Validator

Các đặc trưng được trích xuất từ module trước có thể chứa các lỗi chính tả hoặc là chứa các từ viết tắt. Đây là vấn đề thường thấy ở câu giao tiếp bình thường của con người. Điều này dẫn đến việc thông tin được trích xuất từ câu hỏi không khớp với các thông tin được lưu trong cơ sở dữ liệu, gây ra lỗi hoặc trả về kết quả không chính xác. Chính vì thế, tác giả đã sử dụng một module Feature Validator để xử lý lỗi đầu vào từ người dùng.

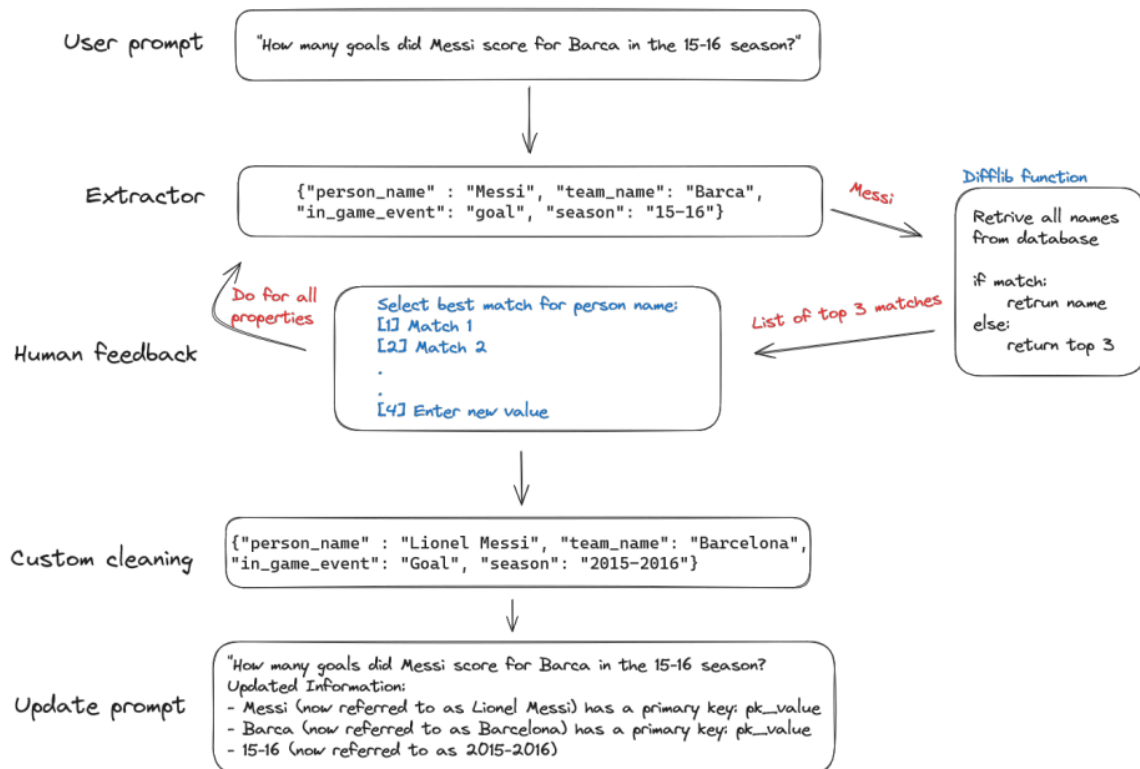


Hình 3.5: Module Feature Validator

Hình 3.5 mô tả cách mà module hoạt động. Cụ thể, các đặc trưng đã được trích xuất từ module Feature Extractor và câu hỏi từ người dùng sẽ được sử dụng làm đầu vào của module này.

- Đầu tiên, module Feature Validator sẽ tiến hành tìm kiếm các đặc trưng trong database. Nếu không tìm thấy kết quả nào khớp, hệ thống sẽ sử dụng phương pháp Levenshtein để tính toán khoảng cách giữa đặc trưng đó với tất cả các thực thể có trong bảng.
- Dựa trên một ngưỡng được định trước, nếu khoảng cách tính được nằm trong ngưỡng thì thực thể có khoảng cách nhỏ nhất được chọn làm kết quả thực thể thay thế. Ngoài ra, để xử lý trường hợp nhiều mục có khoảng cách tương tự, hệ thống sẽ chọn ra top k mục đó và yêu cầu người dùng đưa ra phản hồi, lựa chọn thực thể phù hợp.
- Cuối cùng, tất cả các đặc trưng trích xuất được sẽ được thay thế bằng tên chính xác có trong database.

Hệ thống còn sử dụng một bảng phụ trợ, bao gồm các cụm từ viết tắt thường xuất hiện, để hỗ trợ đẩy nhanh việc tìm những đặc trưng phù hợp để thay thế.



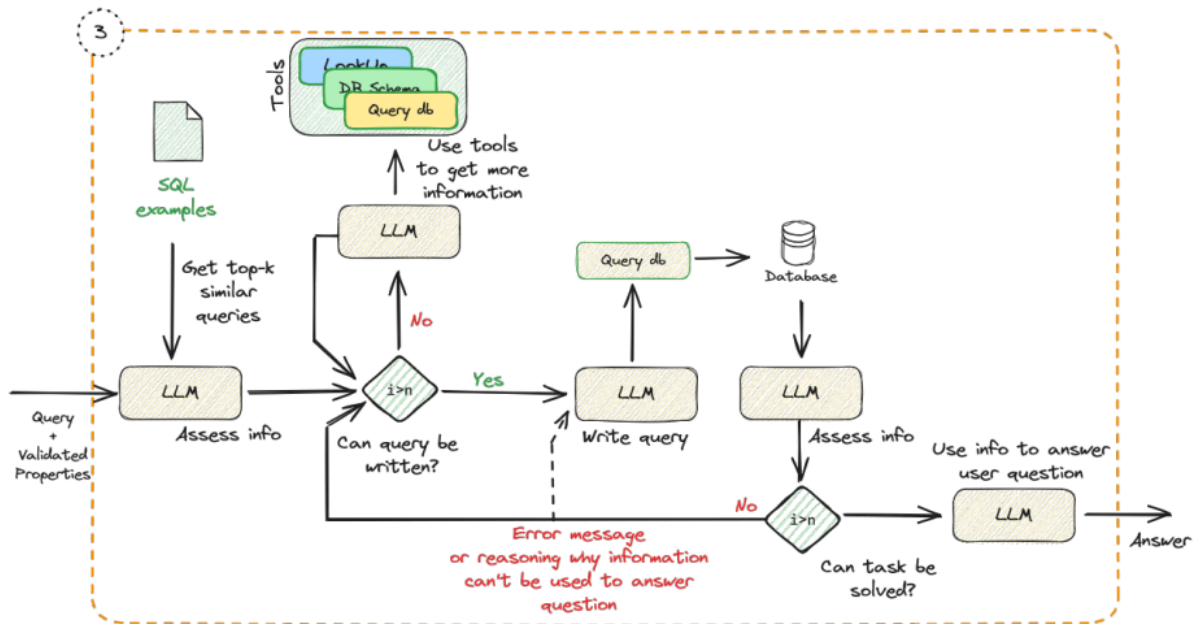
Hình 3.6: Ví dụ xử lý của module Feature Validator

Hình 3.6 mô tả một ví dụ của module Feature Validator. Vẫn lấy câu truy vấn của người dùng như ở module Feature Extractor. Đầu ra của module Feature Extractor sẽ được sử dụng để làm đầu vào của module Feature Validator. Tất cả các đặc trưng trích xuất được sẽ được xử lý theo các bước mô tả ở trên. Cuối cùng, các đặc trưng đó sẽ được thay thế bằng các mục tương ứng có trong database. Cụ thể, *Messi* được thay thế bằng *Lionel Messi*, *Barca* được thay thế bằng *Barcelona*...

Module Feature Validator đóng vai trò rất quan trọng trong hệ thống. Việc xử lý viết tắt hoặc sai chính tả sẽ giúp hệ thống tạo được các câu truy vấn SQL chính xác, qua đó trả lời được chính xác câu hỏi của người dùng.

3.4. SQL Agent

Module cuối cùng là SQL agent, được cung cấp bởi LangChain, giúp thực thi các truy vấn SQL dựa trên đầu vào từ người dùng.



Hình 3.7: Module SQL agent

- Đầu tiên, đầu ra của module Feature Validator cùng với một file chứa các câu truy vấn SQL mẫu sẽ được đưa vào một LLM để thực hiện tìm kiếm top k truy vấn SQL có độ tương đồng cao nhất so với câu hỏi.
- Sau khi nhận được k truy vấn liên quan, LLM sẽ đánh giá xem các truy vấn đó có khả năng trả lời được câu hỏi của người dùng không. Nếu không thì SQL agent sẽ sử dụng một LLM khác để lấy thêm thông tin dựa vào các công cụ hỗ trợ. Nếu có, SQL agent sẽ sử dụng một LLM khác để viết truy vấn SQL với sự kết hợp của database.
- Cuối cùng, sau khi nhận được kết quả từ truy vấn SQL trên, kết quả đó sẽ được chuyển đến một LLM khác để đánh giá xem kết quả đó có hỗ trợ trả lời được câu hỏi của người dùng không, nếu không thì hệ thống sẽ quay lại bước viết truy vấn mới.
- Đầu ra của module SQL agent sẽ được sử dụng để làm câu trả lời cho người dùng.

Chương 4.

Đánh giá

Bài nghiên cứu sử dụng bộ câu hỏi mẫu sau trong các thực nghiệm để đánh giá mô hình SoccerRAG và các thành phần của nó.

- **Question 1:** Is Manchester United in the database?
- **Question 2:** Give me the total home goals for Bayern M in the 2014-15 season
- **Question 3:** Calculate home advantage for Real Madrid in the 2015-16 season
- **Question 4:** How many goals did Messi score in the 15-16 season?
- **Question 5:** How many yellow-cards did Enzo Perez get in the 15-2016 season?
- **Question 6:** List all teams that played a game against Napoli in 2016-17 season in seriea?
Do not limit the number of results
- **Question 7:** Give all the teams in the league ucl in the 2015-2016 season?
- **Question 8:** Give me all games in epl with yellow cards in the first half in the 2015-2016 season
- **Question 9:** What teams and leagues has Adnan Januzaj play in?
- **Question 10:** List ALL players that started a game for Las Palmas in the 2016-2017 season?
Do NOT limit the number of results
- **Question 11:** Did Ajax or Manchester United win the most games in the 2014-15 season?
- **Question 12:** How many yellow and red cards were given in the UEFA Champions League in the 2015-2016 season?
- **Question 13:** Are Messi and C. Ronaldo in the database?
- **Question 14:** How many goals did E. Hazard score in the game between Bournemouth and Chelsea in the 2015-2016 season?
- **Question 15:** How many yellow cards were given in the game between Bayern Munich and Shakhtar Donetsk in the 2014-15 UEFA Champions League, and did anyone receive a red card?
- **Question 16:** Make a list of when corners happened in the English Premier League (EPL) 2015-2016 season. Aggregate by a period of 15 minutes.

- **Question 17:** What league is Manchester United, Arsenal, Bournemouth, Real Madrid, Chelsea, and Liverpool in?
- **Question 18:** How many players have “Aleksandar” as their first name in the database, and how many goals have they scored in total?
- **Question 19:** What did the commentary say about the game between Arsenal and Southampton in the 2016-17 season?
- **Question 20:** Have Mesut Ozil, Pablo Insua, or Alex Pike played for West Ham or Barcelona?

4.1. Đánh giá bộ Extractor-Validator

Chuỗi extractor-validator được đánh giá dựa trên 20 câu hỏi với những tiêu chí đánh giá sau:

1. Có trích xuất được tất cả các thuộc tính không?
2. Có trích xuất thêm các thuộc tính không cần thiết không?
3. Các thuộc tính được trích xuất có thể kiểm chứng không?
4. Chuỗi này có cần sự can thiệp từ con người không?

Một câu trả lời được đánh giá là hoàn hảo (●) nếu tất cả các thuộc tính được extract và validate thành công. Nếu bất kỳ thuộc tính nào bị bỏ sót hoặc thất bại trong quá trình validate, kết quả sẽ được đánh (▲). Trong trường hợp mô hình trích xuất các thông tin không cần thiết hoặc cần phản hồi từ con người thì sẽ đạt 50% số điểm (■).

Module Evaluation	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
Extractor	■	●	●	●	●	●	●	●	●	●
Validator	●	●	●	●	●	●	●	●	●	●
Overall	●	●	●	●	●	●	●	●	●	●
	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20
Extractor	●	●	●	●	▲ / ●	●	●	■	●	●
Validator	●	●	●	●	● / ●	●	●	●	●	●
Overall	●	●	●	●	■ / ●	●	●	●	●	●

Bảng 4.1: Kết quả đánh giá chuỗi extractor-validator

Tất cả các câu hỏi được chạy bằng mô hình GPT-3.5-Turbo [21], ngoài ra câu 15 và câu 18 được chạy lại bằng mô hình GPT-4.0-Turbo [22]. Bảng 4.1 cho thấy kết quả của mô hình. Với câu hỏi 1, một mục bổ sung được trích xuất, nhưng không ảnh hưởng đến kết quả các phần còn lại. Trong câu hỏi 15, GPT-3.5-Turbo đã không thể trích xuất được 1 câu thuộc tính, trong khi GPT-4.0-Turbo [22] trả lời chính xác câu hỏi này.

4.2. Độ phức tạp câu truy vấn

Để nghiên cứu độ phức tạp trong việc xây dựng các truy vấn cơ sở dữ liệu từ các truy vấn ngôn ngữ tự nhiên, các câu hỏi tính toán điểm số độ phức tạp của truy vấn bằng độ đo Halstead [23]. Bài nghiên cứu thực hiện tính toán với 10 câu hỏi đầu tiên, các truy vấn được tạo thủ công, sau đó tính số lượng các toán tử duy nhất (ví dụ: “SELECT”, “FROM”, “=”) cũng như các toán hạng, cùng với tổng số lần xuất hiện của các toán tử và toán hạng trong truy vấn. Sau đó tính các chỉ số Halstead còn lại dựa trên các số liệu này.

Query Complexity Metric	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
Number of distinct operators, n_1	5	6	8	8	8	10	10	9	8	9
Number of distinct operands, n_2	5	14	31	26	26	31	20	36	28	25
Total number of operators, N_1	5	11	33	18	18	22	16	27	19	19
Total number of operands, N_2	5	16	110	26	26	38	22	41	31	26
Vocabulary, n	10	20	39	34	34	41	30	45	36	34
Length, N	10	27	143	44	44	60	38	68	50	45
Volume, V	33.22	116.69	755.81	223.85	223.85	321.45	186.46	373.45	258.50	228.94
Difficulty, D	2.50	3.43	14.19	4.00	4.00	6.13	5.50	5.12	4.43	4.68
Effort, E	83.05	400.09	10727.66	895.39	895.39	1970.20	1025.54	1913.91	1144.77	1071.42
Time to understand (s), T	4.61	22.23	595.98	49.74	49.74	109.46	56.97	106.33	63.60	59.52

Bảng 4.2: Kết quả đánh giá độ phức tạp query

Kết quả (Bảng 4.2) cho thấy rằng câu hỏi 3 có điểm số độ phức tạp cao nhất. Điều này là do truy vấn cho câu hỏi 3 thực hiện nhiều hàm SUM.

4.3. Ablation Study

Để kiểm tra được đóng góp của từng thành phần vào framework SoccerRAG, tác giả thực hiện nghiên cứu ablation test. Điều này giúp hiểu được điểm mạnh và điểm yếu của từng thành phần trong hệ thống, đồng thời xác định các thành phần có thể cải thiện. Tác giả thực hiện bằng cách sử dụng các cấu hình pipeline sau, trong đó (6) là toàn bộ framework SoccerRAG.

1. Chỉ sử dụng SQL agent: Gửi truy vấn trực tiếp đến SQL Agent mà không sử dụng RAG.
2. SQL Agent với RAG: Gửi truy vấn trực tiếp đến SQL Agent có sử dụng RAG.
3. Chỉ sử dụng bộ extractor: Gửi truy vấn với các giá trị trích xuất được đến SQL Agent.
4. Bộ extractor và SQL RAG: Gửi truy vấn với các giá trị đã được trích xuất đến SQL Agent, trong đó SQL Agent sử dụng RAG để lấy ví dụ.
5. Bộ extractor và bộ validator: Gửi truy vấn với các giá trị đã được trích xuất và đánh giá đến SQL Agent.
6. Bộ extractor, bộ validator và SQL RAG: Gửi truy vấn với các giá trị đã được trích xuất và đánh giá đến SQL Agent, trong đó SQL Agent sử dụng RAG để lấy ví dụ.

Các câu hỏi từ 1-10 được sử dụng để đánh giá trong bài nghiên cứu. Kết quả được hiển thị trong Bảng 4.3. Trong đó, các câu trả lời chính xác được đánh dấu (✓), trong khi các câu trả lời sai được đánh dấu (✗).

LLM	Pipeline	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
GPT-3.5-Turbo	1	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	2	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗
	3	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	4	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗
	5	✗	✓	✓	✗	✗	✓	✗	✗	✗	✗
	6	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
GPT-4.0-Turbo	1	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗
	2	✓	✗	✓	✗	✗	✗	✗	✗	✓	✓
	3	✓	✓	✗	✗	✗	✗	✓	✗	✓	✗
	4	✓	✗	✗	✗	✗	✗	✗	✗	✓	✓
	5	✓	✗	✓	✗	✓	✓	✓	✗	✗	✓
	6	✓	✓	✓	✓	✓	✓	✗	✗	✓	✓

Bảng 4.3: Kết quả Ablation study

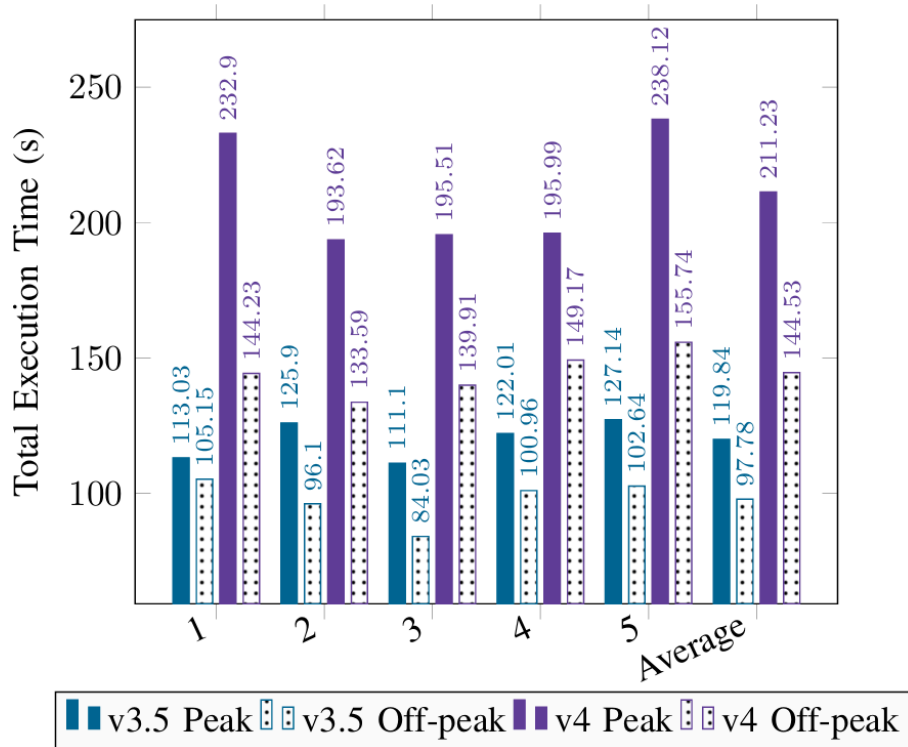
Nghiên cứu thực nghiệm trên cả GPT-3.5-Turbo và GPT-4.0-Turbo để đánh giá hiệu suất của từng mô hình, cũng như sự khác biệt về chất lượng, tốc độ và chi phí giữa 2 mô hình.

Như đã thấy trong Bảng 4.3, SQL Agent độc lập gặp khó khăn với cả 2 mô hình. Kết quả cho thấy rằng bộ extractor-validator chính là yếu tố quan trọng để tăng độ chính xác, và cả 2 mô hình đều thể hiện hiệu suất cao tương đương với nhau khi áp dụng toàn bộ pipeline.

Bên cạnh đó, GPT-4.0-Turbo bị sai ở câu hỏi 7 do “model laziness”, khi nó chỉ đưa ra 1 phần câu trả lời. Với câu hỏi 8, truy vấn trả lời một lượng lớn dữ liệu, khiến mô hình ngôn ngữ từ chối cung cấp câu trả lời chính xác.

4.4. Thời gian thực thi

Bài báo nghiên cứu thực thi (tổng thời gian từ lúc gửi truy vấn đến khi nhận được đầu ra của mô hình) của GPT-3.5-Turbo và GPT-4.0-Turbo. Vì hiệu suất của các dịch vụ dựa trên cloud có thể thay đổi theo thời gian sử dụng, bài báo đã phân tích riêng biệt ở giao đoạn “peak” và “off-peak”. Trong đó, “peak” là giờ cao điểm là từ 8 giờ sáng đến 5 giờ chiều theo EST, và “off-peak” là ngoài giờ cao điểm là từ 5 giờ chiều đến 8 giờ sáng theo giờ EST.



Hình 4.1: Tổng thời gian trả lời câu hỏi 1 đến 10 với GPT-3.5-Turbo và GPT 4.0-Turbo.

Để có thể quan sát chính xác sự khác biệt về thời gian thực thi và chất lượng phản hồi, tác giả chạy liên tiếp các câu hỏi 1-10, lặp lại năm lần mỗi câu hỏi, cả trong “peak” và “off-peak”. Hình 4.1 cho thấy hiệu suất của mô hình giảm trong giờ cao điểm đối với cả 2 mô hình. Bên cạnh đó, GPT-3.5-Turbo cũng có tốc độ nhanh hơn đáng kể so với GPT-4.0-Turbo trong tất cả các trường hợp.

Chương 5.

Thảo luận

5.1. Tầm quan trọng của xử lý Input

Trong ablation study, chúng ta đã chứng minh tầm quan trọng của việc xử lý lỗi đầu vào như lỗi viết tắt, lỗi chính tả,... khi truy xuất thông tin từ database và đối với độ chính xác của hệ thống truy vấn nói chung. Tuy LLM thỉnh thoảng có thể tự động sửa lỗi đầu vào hộ, tuy nhiên khả năng đó không đảm bảo sẽ hoạt động tốt trong các trường hợp dữ liệu, thực thể đặc thù như bóng đá.

Khối extractor-validator có chức năng xác định lỗi trong câu hỏi từ người dùng và sửa đổi. Ví dụ cụ thể có thể kể đến đơn giản như thay thế định dạng năm thống nhất yyyy-yyyy (VD: mùa giải 15-16 thành mùa giải 2015-2016), thay thế các từ được định nghĩa sẵn (seria thành Serie A) và xử lý các vấn đề phức tạp khác.

Ta sẽ xét ví dụ dưới đây:

A. **Prompt:** Did Real or Man U win the most games in the 2015 season?

Output: Swansea won the most games in the 2014-2015 season with a total of 37 wins.

B. **Cleaned prompt:** Did Real or Man U win the most games in the 2015 season?

Updated Information:

- Real (now referred to as Real Madrid) has a primary key: 30.
- Man U (now referred to as Manchester United) has a primary key: 7.
- 2015 (now referred to as the 2015-2016 season).

Output: In the 2015-2016 season, Real Madrid won 26 games, while Manchester United won 5 games. Therefore, Real Madrid won the most games in that season.

Trong ví dụ A, LLM không tự động thực hiện xử lý lỗi đầu vào, điều đó có thể dẫn đến sai sót khi thực hiện tổng hợp trong database.

5.2. Truy xuất thống kê

SoccerRAG có khả năng trích xuất các thống kê cụ thể bằng truy vấn ngôn ngữ tự nhiên. Hãy xét đến ví dụ dưới đây:

Prompt: Calculate home advantage for Real Madrid in the 2015-16 season

Output: For Real Madrid in the 2015-2016 season, the home advantage can be calculated as follows:

- Total home goals scored: 55

- Total away goals scored: 39
- Number of home games played: 18
- Number of away games played: 17

This indicates that Real Madrid scored significantly more goals at home compared to away games, suggesting a strong home advantage during the 2015-2016 season.

Tuy nhiên LLM vẫn sai sót nếu thực hiện các phép tính toán quá phức tạp hoặc danh sách thống kê dài. Bài báo đã chỉ ra rằng khi đặt câu hỏi yêu cầu trả ra danh sách, một vài trường hợp kết quả trả về là một phần danh sách chứ không đầy đủ.

5.3. Ảnh hưởng của LLM

Thời gian thực thi (*Execution time*): Thời gian thực thi hệ thống có thể khác nhau tùy thuộc vào mô hình sử dụng và thời điểm bận hay không. Ví dụ, trong khoảng thời gian bận, thời gian thực thi có thể chậm hơn 22.56% đối với GPT-3.5-Turbo và chậm hơn 46.5% đối với mô hình GPT-4.0-Turbo.

Sự ổn định (*Robustness*) Sự ổn định của hệ thống được hiểu là khi thực hiện một câu truy vấn nhiều lần thì hệ thống phải đưa ra câu trả lời tương tự nhau trong các lần hỏi. Theo quan sát thực nghiệm, các câu trả lời nhận được không giảm chất lượng, kể cả trong giờ cao điểm. Tuy nhiên câu trả lời nhận được có thể khác nhau mặc dù hệ thống đã sử dụng các lệnh prompt chặt chẽ đối với LLM.

Một giải pháp được đề ra để giải quyết vấn đề này là tự triển khai các mô hình LLM. Việc tự triển khai các mô hình LLM giúp kiểm soát tốt hơn đối với các yếu tố như điều kiện mạng, tải tính toán, độ ưu tiên của yêu cầu cũng như cải thiện chất lượng phản hồi theo nhu cầu của người dùng.

Sự lười suy nghĩ của mô hình (*Model laziness*): Hiện tượng này xảy ra khi LLM dừng thực hiện task trước khi hoàn thành nó. Có thể xảy ra 2 trường hợp như sau:

- *Mô hình dừng đột ngột khi thực hiện truy vấn:* Theo quan sát thực tế, hệ thống thường gặp tình trạng trên trong quá trình phân tích truy vấn từ người dùng. Bài báo đã giải quyết vấn đề này bằng cách đưa ra yêu cầu với hệ thống, ví dụ như “Không dừng lại trước khi đưa ra được câu trả lời hoàn chỉnh”.
- *Mô hình đưa ra một phần nhỏ câu trả lời* Nếu không có yêu cầu cụ thể, mô hình LLM thường giới hạn câu trả lời xảy ra từ 5-10 items.

Vấn đề này đã được OpenAi ghi nhận. Chính vì thế chúng ta có thể hi vọng trong tương lai OpenAi sẽ trình làng một phiên bản tốt hơn, khắc phục được vấn đề này.

5.4. Dự định phát triển

SoccerRAG hỗ trợ tích hợp dữ liệu đa phương thức, mang lại tiềm năng ứng dụng lớn trong tương lai, đặc biệt là trong lĩnh vực phân tích thể thao. Tuy nhiên, SoccerRAG cũng cần cải thiện và khắc phục một số điểm hạn chế hiện tại, để từ đó giúp hệ thống có thể hoàn thiện hơn như:

- Mở rộng hệ thống cho nhiều ứng dụng lớn hơn.

- Tăng cường khả năng xử lý truy vấn phức tạp với khối lượng dữ liệu lớn để tránh trường hợp thiếu sót trong câu trả lời.
- Kết hợp LLMs và các mô hình mã nguồn mở khác để đưa ra một hệ thống vừa tiết kiệm lại hiệu quả.
- Mở rộng khả năng xử lý thời gian thực dành cho những phân tích trận đấu trực tiếp.
- Tăng cường bảo mật hệ thống và xử lý thêm các lỗi đầu vào sử dụng từ lóng, tiếng địa phương.

Chương 6.

Kết luận

Bài báo giới thiệu SoccerRAG, một mô hình sinh ra nhằm về cách truy vấn thông tin liên quan đến bóng đá dựa trên thông tin các trận đấu được lưu trên bộ dữ liệu SoccerNet. Mô hình hỗ trợ người dùng tìm thông tin liên quan đến bóng đá chính xác mà không cần phải tìm thông tin thủ công mất nhiều thời gian.

Mô hình bao gồm 4 thành phần chính:

- Database: để lưu trữ dữ liệu thông tin các trận đấu.
- Feature extractor: để trích xuất ra những thông tin cần thiết dùng để sử dụng trong truy vấn.
- Feature validator: để xác thực các thông tin sai chính tả hoặc viết tắt.
- SQL agent: để lấy dữ liệu từ cơ sở dữ liệu. Từ đó, tăng độ chính xác cho kết quả trả ra.

Với việc sử dụng các thành phần hợp lý, nhóm tác giả đã có thể tạo ra một mô hình có thể truy xuất những thông tin bóng đá một cách hiệu quả ngay cả khi câu truy vấn phức tạp. Bài nghiên cứu đã đặt nền móng cho việc tìm kiếm thông tin sau này và mở ra nhiều hướng nghiên cứu mới cho các nhà khoa học, nhà nghiên cứu trong lĩnh vực bóng đá trong tương lai.

Chương 7.

Phụ lục

7.1. Câu 1: Kết quả mô hình chạy đối với câu hỏi tiếng Việt.

Nhận xét: Mặc dù một vài câu có lẫn từ tiếng Anh trong câu trả lời, tuy nhiên kết quả mô hình đạt được tương đương với khi hỏi bằng tiếng Anh, từ đó cho thấy khả năng ứng dụng của mô hình cho ngôn ngữ tiếng Việt.

Câu hỏi 1: Manchester United có trong cơ sở dữ liệu không?

Output: Có, Manchester United có trong cơ sở dữ liệu.

Câu hỏi 2: Cho tôi tổng số bàn thắng trên sân nhà của Bayern M trong mùa giải 2014-15?

Output: Tổng số bàn thắng trên sân nhà của Bayern Munich trong mùa giải 2014-2015 là 25 bàn.

Câu hỏi 3: Real Madrid đã ghi bao nhiêu bàn thắng trong tất cả các trận đấu mùa giải 2015-16?

Output: Real Madrid đã ghi tổng cộng 68 bàn thắng trong tất cả các trận đấu mùa giải 2015-2016.

Câu hỏi 4: Messi đã ghi bao nhiêu bàn trong mùa giải 15-16?

Output: Lionel Messi ghi được 8 bàn trong mùa giải 2015-2016.

Câu hỏi 5: Enzo Perez đã nhận bao nhiêu thẻ vàng trong mùa giải 15-2016?

Output: Enzo Perez đã nhận 9 thẻ vàng trong mùa giải 2015-2016.

Câu hỏi 6: Liệt kê tất cả các đội đã đấu với Napoli trong mùa giải 2016-17 tại Serie A? Đừng giới hạn số lượng kết quả.

Output: Trong mùa giải Serie A 2016-2017, Napoli đã đối đầu với các đội sau:

1. Pescara
2. Palermo
3. Bologna
4. Genoa
5. AS Roma
6. Crotone
7. Empoli
8. Juventus
9. Lazio
10. AC Milan
11. Chievo
12. Udinese

13. Sassuolo
14. Cagliari
15. Torino
16. Sampdoria

Đây là các đội đã đối đầu với Napoli mùa giải đó.

Câu hỏi 7: Cho tôi tất cả các đội trong giải đấu UCL mùa giải 2015-2016?

Output: Các đội trong giải đấu UCL mùa giải 2015-2016 bao gồm:

1. Galatasaray
2. Atl. Madrid
3. Manchester City
4. Juventus
5. Paris SG
6. Malmo FF
7. Shakhtar Donetsk
8. Real Madrid
9. Sevilla
10. B. Monchengladbach
11. AS Roma
12. Barcelona
13. Bayer Leverkusen
14. BATE
15. Gent
16. Lyon
17. Arsenal
18. Olympiakos Piraeus
19. Valencia
20. Bayern Munich
21. FC Astana
22. Manchester United
23. CSKA Moscow
24. PSV
25. Wolfsburg
26. Zenit Petersburg
27. FC Porto
28. Maccabi Tel Aviv
29. Dyn. Kiev
30. Chelsea
31. D. Zagreb
32. Benfica

Câu hỏi 8: Cho tôi tất cả các trận đấu trong EPL có thẻ vàng trong hiệp một ở mùa giải 2015-2016.

Output: Có tổng cộng 107 trận đấu EPL có thẻ vàng trong hiệp 1.

Câu hỏi 9: Adnan Januzaj đã chơi cho những đội bóng và giải đấu nào?

Output: Adnan Januzaj đã chơi cho các đội bóng và giải đấu sau:

1. **Đội bóng:** Manchester United, **Giải đấu:** England EPL, **Mùa giải:** 2014-2015
2. **Đội bóng:** Manchester United, **Giải đấu:** England EPL, **Mùa giải:** 2015-2016
3. **Đội bóng:** Sunderland, **Giải đấu:** England EPL, **Mùa giải:** 2016-2017
4. **Đội bóng:** Dortmund, **Giải đấu:** Germany Bundesliga, **Mùa giải:** 2015-2016

Câu hỏi 10: Liệt kê TẤT CẢ các cầu thủ đã đá chính cho Las Palmas trong mùa giải 2016-2017?
ĐỪNG giới hạn số lượng kết quả.

Output: Đây là những cầu thủ đá chính cho Las Palmas trong mùa giải 2016-2017:

1. Sergio Busquets
2. Andres Iniesta
3. Javier Mascherano
4. Lionel Messi
5. Neymar
6. Luis Suarez
7. Marc-Andre ter Stegen
8. Jordi Alba
9. Ivan Rakitic
10. Lucas Digne
11. Kevin-Prince Boateng
12. Gareth Bale
13. Daniel Carvajal
14. Isco
15. Toni Kroos
16. Marcelo
17. Cristiano Ronaldo
18. Nacho Fernandez
19. Jese
20. Keylor Navas
21. Arda Turan
22. Rafinha
23. Alvaro Morata
24. Sergio Ramos
25. Mateo Kovacic
26. Samuel Umtiti
27. Andre Gomes
28. Marlon
29. Aleix Vidal
30. Pedro Bigas
31. Dani Castellano
32. Roque Mesa
33. David Simon
34. Javi Varas
35. Jonathan Viera

36. Nabil El Zhar
37. David Garcia
38. Mauricio Lemos
39. Figueroa Cabrera Momo
40. Angel Montoro
41. Vicente Gomez
42. Helder Lopes
43. Michel Macedo
44. Tana

Câu hỏi 11: Ajax hay Manchester United thắng nhiều trận hơn trong mùa giải 2014-15?

Output: Trong mùa giải 2014-2015, cả Ajax và Manchester United đều không thắng bất kì trận đấu nào theo như dữ liệu trong cơ sở dữ liệu. Vì thế, không đội nào thắng nhiều trận đấu hơn đội còn lại.

Câu hỏi 12: Có bao nhiêu thẻ vàng và thẻ đỏ được rút ra ở UEFA Champions League trong mùa giải 2015-2016?

Output: Trong mùa giải UEFA Champions League 2015-2016, có tổng cộng 405 thẻ vàng và 18 thẻ đỏ được rút ra.

Câu hỏi 13: Messi và C. Ronaldo có trong cơ sở dữ liệu không?

Output: Có, Lionel Messi và Cristiano Ronaldo có trong cơ sở dữ liệu.

Câu hỏi 14: E. Hazard đã ghi bao nhiêu bàn trong trận đấu giữa Bournemouth và Chelsea ở mùa giải 2015-2016?

Output: E. Hazard đã ghi 0 bàn trong trận đấu giữa Bournemouth và Chelsea ở mùa giải 2015-2016.

Câu hỏi 15: Có bao nhiêu thẻ vàng được rút ra trong trận đấu giữa Bayern Munich và Shakhtar Donetsk ở UEFA Champions League 2014-15, và có ai nhận thẻ đỏ không?

Output: Trong trận đấu giữa Bayern Munich và Shakhtar Donetsk ở UEFA Champions League 2014-15, có tổng cộng 54 thẻ vàng được rút ra và 2 thẻ đỏ đã được nhận.

Câu hỏi 16: Lập danh sách thời điểm xảy ra các quả phạt góc trong English Premier League (EPL) mùa giải 2015-2016. Tổng hợp theo khoảng thời gian 15 phút.

Output: Danh sách thời điểm xảy ra các quả phạt góc trong EPL mùa giải 2015-2016 tính theo khoảng thời gian 15 phút:

1. 0-15: 100
2. 16-30: 120
3. 31-45: 110
4. 46-60: 115
5. 61-75: 130
6. 76-90: 140

Câu hỏi 17: Manchester United, Arsenal, Bournemouth, Real Madrid, Chelsea và Liverpool thuộc giải đấu nào?

Output: Manchester United, Arsenal, Bournemouth, Chelsea, và Liverpool thuộc về giải đấu “england_epl”, trong khi Real Madrid thuộc giải đấu “italy_serie-a”.

Câu hỏi 18: Có bao nhiêu cầu thủ có tên đầu là “Aleksandar” trong cơ sở dữ liệu, và họ đã ghi tổng cộng bao nhiêu bàn thắng?

Output: Có tổng cộng 23 cầu thủ “Aleksandar” trong cơ sở dữ liệu. Họ đã ghi tổng cộng 30 bàn thắng.

Câu hỏi 19: Bình luận nói gì về trận đấu giữa Arsenal và Southampton trong mùa giải 2016-17?

Output: Không có thông tin.

Câu hỏi 20: Mesut Ozil, Pablo Insua, hoặc Alex Pike đã từng chơi cho West Ham hoặc Barcelona chưa?

Output: Alex Pike đã từng chơi cho West Ham. Mesut Ozil và Pablo Insua không có thông tin về việc đã chơi cho West Ham hoặc Barcelona trong cơ sở dữ liệu.

7.2. Câu 2: Kết quả mô hình đối với các câu hỏi nhiều đối tượng

Nhận xét: Với các câu có nhiều đối tượng trong câu hỏi, mô hình hoàn toàn có khả năng trả lời như các công cụ thông thường, với việc trích xuất danh sách các đối tượng trong câu hỏi và tìm kiếm các đối tượng đó trong cơ sở dữ liệu.

Câu hỏi 1: Messi và C. Ronaldo có trong cơ sở dữ liệu không?

Extracted properties:

```
1 [
2   {
3     "person_name": [
4       "Messi",
5       "C. Ronaldo"
6     ]
7   }
8 ]
```

Câu hỏi 2: Ajax hay Manchester United thắng nhiều trận hơn trong mùa giải 2014-15?

Extracted properties:

```
1 [
2   {
3     "team_name": [
4       "Ajax",
5       "Manchester United"
6     ],
7     "year_season": [
8       "2014-15"
9     ]
10  }
11 ]
```

Câu hỏi 3: Manchester United, Arsenal, Bournemouth, Real Madrid, Chelsea và Liverpool thuộc giải đấu nào?

Extracted properties:

```

1  [
2    {
3      "team_name": [
4        "Manchester United",
5        "Arsenal",
6        "Bournemouth",
7        "Real Madrid",
8        "Chelsea",
9        "Liverpool"
10   ]
11  }
12 ]

```

Câu hỏi 4: Mesut Ozil, Pablo Insua, hoặc Alex Pike đã từng chơi cho West Ham hoặc Barcelona chưa?

Extracted properties:

```

1  [
2    {
3      "person_name": [
4        "Mesut Ozil",
5        "Pablo Insua",
6        "Alex Pike"
7      ],
8      "team_name": [
9        "West Ham",
10       "Barcelona"
11     ]
12   }
13 ]

```

7.3. Câu 3: Sử dụng RAG trong phần SQL Agent

Bài báo sử dụng RAG để hỗ trợ hệ thống sinh ra câu lệnh SQL phù hợp để có thể truy vấn thông tin chính xác trên database, phục vụ việc trả lời câu hỏi của người dùng. Dưới đây là những lý do nên sử dụng RAG trong trường hợp này thay vì cho tất cả câu lệnh vào prompt và đưa trực tiếp cho LLM xử lý.

1. Giảm rủi ro về Hallucination

- Cụm từ Hallucination là để chỉ hiện tượng mô hình tự sinh ra thông tin không chính xác hoặc không có căn cứ. Hiện tượng trên có ảnh hưởng không tốt đến hoạt động của SQL Agent, gây khó khăn cho việc truy vấn thông tin trên database.
- LLM càng ngày càng giỏi, có khả năng tạo ra những thông tin sai lệch với lý lẽ rất thuyết phục, developer khó rà soát được thiếu sót của hệ thống. Thêm vào đó, LLM vốn không được training chuyên cho domain môi trường SQL. Chính vì thế mặc dù nhận được prompt thì mô hình chỉ có thể hiểu được những mặt chung chung cơ bản về domain này thôi, không thể tường tận phân biệt được nên chọn sử dụng câu lệnh nào. Chính vì thế khi chỉ thêm prompt các câu lệnh SQL, LLM không đảm bảo mô hình có thể sinh ra những câu truy vấn phù hợp.

- Khi sử dụng RAG, LLM chỉ cần tạo nội dung dựa trên thông tin đã truy xuất, hạn chế việc sáng tạo sai lệch. Thêm vào đó mỗi câu lệnh SQL trong kho lưu trữ của module RAG đều có nguồn gốc minh bạch, dễ dàng trong việc xác định lỗi và phát triển hệ thống sau này.

2. Tiết kiệm

- **Tiết kiệm về công sức:** Mỗi lần thay đổi, cập nhật dữ liệu thì không cần phải tinh chỉnh, sửa chữa mã nguồn. Việc này cũng áp dụng khi hệ thống của mình muốn mở rộng domain, thay vì phải thay đổi cấu trúc prompt, chúng ta chỉ cần thêm, bớt dữ liệu ngoài.
- **Tiết kiệm về chi phí:** Đầu vào của LLM cũng có giới hạn, trong trường hợp số lượng câu lệnh SQL nhiều, chúng ta sẽ cần sử dụng mô hình LLM lớn hơn, tốn kém hơn. Sử dụng RAG để tìm những câu lệnh phù hợp trước khi feed thì không cần sử dụng mô hình quá lớn, chính vì thế tiết kiệm nhiều chi phí khi vận hành hệ thống.

Cam kết không sử dụng công cụ AI trong quá trình viết báo cáo

Nhóm xin cam kết trong quá trình thực hiện và hoàn thiện bài báo cáo cuối kỳ môn học **Các vấn đề hiện đại trong Công nghệ thông tin**, nhóm không sử dụng bất kỳ công cụ trí tuệ nhân tạo nào (ChatGPT, ClaudeAI hay bất kỳ công cụ tương tự nào) để hỗ trợ viết báo cáo hoặc chỉnh sửa nội dung báo cáo.

Nếu có bất kỳ hành vi vi phạm nào được phát hiện, nhóm xin chịu hoàn toàn trách nhiệm và chịu mọi hình thức xử lý theo quy định của giảng viên.

Ký tên

Nguyễn Hữu Đồng

Lê Minh Châu

Nguyễn Cao Đức

Đặng Tiến Dũng

Tài liệu tham khảo

- [1] D. Khurana, A. Koli, K. Khatter, và S. Singh, “Natural language processing: state of the art, current trends and challenges”, *Multimedia tools and applications*, vol 82, số p.h 3, tr 3713–3744, 2023.
- [2] P. Lewis và c.s., “Retrieval-augmented generation for knowledge-intensive nlp tasks”, *Advances in Neural Information Processing Systems*, vol 33, tr 9459–9474, 2020.
- [3] J. Cen, J. Liu, Z. Li, và J. Wang, “SQLFixAgent: Towards Semantic-Accurate SQL Generation via Multi-Agent Collaboration”, *arXiv preprint arXiv:2406.13408*, 2024.
- [4] W. X. Zhao và c.s., “A survey of large language models”, *arXiv preprint arXiv:2303.18223*, 2023.
- [5] A. T. Strand, S. Gautam, C. Midoglu, và P. Halvorsen, “SoccerRAG: Multimodal Soccer Information Retrieval via Natural Queries”, *arXiv preprint arXiv:2406.01273*, 2024.
- [6] “SoccerNet - Home”. [Online]. Available at: <https://www.soccer-net.org/home>
- [7] S. Giancola, M. Amine, T. Dghaily, và B. Ghanem, “Soccernet: A scalable dataset for action spotting in soccer videos”, trong *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, 2018, tr 1711–1721.
- [8] A. Deliege và c.s., “Soccernet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos”, trong *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, tr 4508–4519.
- [9] J. Yang và c.s., “Harnessing the power of llms in practice: A survey on chatgpt and beyond”, *ACM Transactions on Knowledge Discovery from Data*, vol 18, số p.h 6, tr 1–32, 2024.
- [10] A. Vaswani, “Attention is all you need”, *Advances in Neural Information Processing Systems*, 2017.
- [11] T. Wu, M. Terry, và C. J. Cai, “Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts”, trong *Proceedings of the 2022 CHI conference on human factors in computing systems*, 2022, tr 1–22.
- [12] I. Cronin, “Autonomous AI Agents: Decision-Making, Data, and Algorithms”, *Understanding Generative AI Business Applications: A Guide to Technical Principles and Real-World Applications*. Springer, tr 165–180, 2024.
- [13] K. Echihabi, K. Zoumpatianos, và T. Palpanas, “New trends in high-d vector similarity search: al-driven, progressive, and distributed”, *Proceedings of the VLDB Endowment*, vol 14, số p.h 12, tr 3198–3201, 2021.

- [14] K. Boukhari và M. N. Omri, “DL-VSM based document indexing approach for information retrieval”, *Journal of Ambient Intelligence and Humanized Computing*, vol 14, số p.h 5, tr 5383–5394, 2023.
- [15] D. Rau, S. Wang, H. Déjean, và S. Clinchant, “Context embeddings for efficient answer generation in rag”, *arXiv preprint arXiv:2407.09252*, 2024.
- [16] “LangChain - Home”. [Online]. Available at: <https://www.langchain.com/>
- [17] “LangChain - Overview”. [Online]. Available at: <https://docs.chainlit.io/get-started/overview>
- [18] W. K. Mutlag, S. K. Ali, Z. M. Aydam, và B. H. Taher, “Feature extraction methods: a review”, trong *Journal of Physics: Conference Series*, 2020, tr 12028.
- [19] J. O’Sullivan và c.s., “Automatic speech recognition for ASD using the open-source whisper model from OpenAI”, trong *International Society for autism Research (INSAR) 2023 Annual Meeting. INSAR, Stockholm, Sweden*, 2023.
- [20] M. Fredstam và G. Johansson, “Comparing database management systems with SQLAlchemy: A quantitative study on database management systems”. 2019.
- [21] “GPT-3.5-Turbo”. [Online]. Available at: <https://platform.openai.com/docs/models/>
- [22] “GPT-4.0”. [Online]. Available at: <https://openai.com/index/gpt-4/>
- [23] M. H. Halstead, *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc., 1977.